

## Lab 7 – Shift Reduce Parsing

### Aim

To implement shift reduce parsing for checking whether a given input string is accepted by a given grammar.

### Grammar Used

$E \rightarrow E + E$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow i$

### Algorithm

1. Start the program.
2. Read the input string.
3. Initialize the stack as empty.
4. Shift one symbol from input to stack.
5. After every shift, check whether the top of the stack matches any right-hand side of a production.
6. If matched, reduce it to the corresponding non-terminal.
7. Repeat shift and reduce until the input is completely processed.
8. If the final stack contains only the start symbol, accept the string.
9. Otherwise reject it.
10. Stop.

### C Program

```
#include <stdio.h>
#include <string.h>

char stack[100];
char input[100];
int top = -1;

void push(char c) {
    stack[++top] = c;
    stack[top + 1] = '\0';
}

void reduce() {
    int changed = 1;

    while (changed) {
        changed = 0;

        if (top >= 0 && stack[top] == 'i') {
```

```

        stack[top] = 'E';
        changed = 1;
        printf("Reduce E->i\t\tStack: %s\n", stack);
    }

    if (top >= 2 &&
        stack[top - 2] == 'E' &&
        stack[top - 1] == '+' &&
        stack[top] == 'E') {

        top = top - 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        changed = 1;
        printf("Reduce E->E+E\t\tStack: %s\n", stack);
    }

    if (top >= 2 &&
        stack[top - 2] == 'E' &&
        stack[top - 1] == '*' &&
        stack[top] == 'E') {

        top = top - 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        changed = 1;
        printf("Reduce E->E*E\t\tStack: %s\n", stack);
    }

    if (top >= 2 &&
        stack[top - 2] == '(' &&
        stack[top - 1] == 'E' &&
        stack[top] == ')') {

        top = top - 2;
        stack[top] = 'E';
        stack[top + 1] = '\0';
        changed = 1;
        printf("Reduce E->(E)\t\tStack: %s\n", stack);
    }
}
}

int main() {
    int i;

    stack[0] = '\0';

    printf("Enter input string: ");
    scanf("%s", input);

```

```

printf("\nStack operations:\n");

for (i = 0; input[i] != '\0'; i++) {
    push(input[i]);
    printf("Shift %c\t\tStack: %s\n", input[i], stack);
    reduce();
}

if (strcmp(stack, "E") == 0)
    printf("\nString Accepted\n");
else
    printf("\nString Rejected\n");

return 0;
}

```

## Sample Input

i+i\*i

## Sample Output

```

Stack operations:
Shift i      Stack: i
Reduce E->i  Stack: E
Shift +      Stack: E+
Shift i      Stack: E+i
Reduce E->i  Stack: E+E
Reduce E->E+E Stack: E
Shift *      Stack: E*
Shift i      Stack: E*i
Reduce E->i  Stack: E*i
Reduce E->E*i Stack: E

```

String Accepted

## Result

Thus the shift reduce parser was implemented successfully and the given input string was checked for acceptance.